



React

مقدمه

در این تمرین، توسعه‌ی رابط کاربری (Front-end) اپلیکیشن با بهره‌گیری از فریم‌ورک React صورت خواهد گرفت. با تکمیل این فاز، فرایند گذار به معماری RIA نهایی شده و ساختار برنامه از مدل [SSR](#) به مدل [CSR](#) تغییر می‌یابد.

اپلیکیشن

نیازمندی‌های این برنامه کاملاً یکسان و مشابه با توصیف ارائه شده در تمرین قبلی است. همچنین پیاده‌سازی بخش ارجاع امتیازی خواهد بود.

فریم‌ورک

آشنایی

در واقع، رِاکت یک کتابخانه برای JavaScript محسوب می‌شود. این کتابخانه با گذاشتن یک تگ `<script>` در HTML به سایت ما اضافه می‌شود و ما می‌توانیم از امکانات آن استفاده کنیم. می‌توان از جمله امکانات آن به موارد زیر اشاره کرد:

- ساختار Componentها
- Virtual DOM برای سریع‌تر شدن re-rendering
- Hookها برای مدیریت State و Life Cycle

برای این تمرین خوب است در ابتدا مروری بر مفاهیم JavaScript داشته باشید و سپس وارد رِاکت شوید. JS زبان بزرگی می‌باشد (شامل خود زبان و ترکیب آن با مرورگر و DOM) و عمیق شدن در آن وقت زیادی می‌خواهد. در اینجا

هدف آشنایی کلی با آن، جهت یادگیری ری اکت می باشد. برای این کار می توانید مقدمات «[اینجا](#)» را مطالعه کنید و یا به «[این لینک](#)» برای توضیحات کامل تر مراجعه کنید. همچنین همانند HTML و CSS، سایت [MDN](#) منبع خوبی برای درک جزئی تر توابع است.

برای آشنایی با نحوه کارکرد ری اکت، داکيومنتیشن خود آن بسیار مفید می باشد ([لینک](#)). بخش Quick Start و Tutorial آن را حتماً مطالعه کنید تا با Component ها، State و Hook ها آشنا شوید. بقیه بخش های آن نیز مفید بوده و خوب است به آن ها نگاهی بیندازید؛ مثلاً چالش های مدیریت State را ببینید و با Context آشنا شوید.

روتر

از آنجا که ما با ری اکت یک Single Page Application می سازیم، به طور پیش فرض قابلیت داشتن route های مختلف مانند user/settings/ و navigation بین صفحات را نداریم. برای این کار، فریم ورک [Next.js](#) روتر خود را ارائه کرده است و سایر ابزارها نیز از یک کتابخانه به نام React Router استفاده می کنند.

این کتابخانه در نسخه جدید خود سه mode دارد: Data، Framework و [Normal](#) که ما برای راحتی استفاده از حالت آخر استفاده می کنیم. طریقه استفاده از آن در این [لینک](#) آمده است.

توجه کنید که در SPA ها هیچ گاه صفحه نباید reload شود. تغییر بین صفحات نیز از طریق React Router انجام می شود و هرگز حالتی مانند زدن دکمه Reload مرورگر نداریم. در این راستا باید همیشه به جای تگ `<a>` از کامپوننت [<Link>](#) استفاده کنیم و برای تغییر صفحه در JavaScript از هوک [useNavigate](#) بهره ببریم.

کارهای دیگری مانند دریافت Query Parameter ها یا گرفتن بخش هایی مانند ID از مسیر (URL Path) را نیز می توان با همین کتابخانه انجام داد که در لینک داده شده به آن ها اشاره شده است.

فراخوانی API

برای برقراری ارتباط با سرور و ارسال یا دریافت اطلاعات، می‌توانید از Fetch API یا Axios استفاده کنید. Fetch API به‌صورت پیش‌فرض در استاندارد JavaScript وجود دارد و قابلیت ارسال درخواست‌های HTTP (مانند GET، POST و...) را در مرورگر فراهم می‌کند؛ می‌توانید در «[این لینک](#)» با آن آشنا شوید. از آنجا که سایت ما نباید هنگام ارسال درخواست «قفل» (Freeze) شود، این عملیات به‌صورت Asynchronous (ناهمگام) انجام می‌شوند. بدین منظور از Promise ها (ساختار then) یا همان‌طور که در لینک آمده است، از سینتکس async/await استفاده می‌شود.

پیام‌های سیستم

خطاها و همچنین پیام‌های موفقیت‌آمیز، مانند «درخواست شما با موفقیت انجام شد»، را می‌توانید با استفاده از Toast ها نمایش دهید. برای این کار از کتابخانه React-Toastify استفاده کنید.

نکات پایانی

- این پروژه به صورت انفرادی است.
- برای تحویل پروژه کد های خود (بدونه پوشه node) را zip و در صفحه درس بارگذاری کنید.
- ساختاربندی مناسب پروژه، تمیزی کد و طراحی قابل استفاده مجدد کامپوننت‌ها بخشی از نمره خواهد بود.
- به مفاهیم پایه در React مسلط باشید، در هنگام تحویل از آنها سوال پرسیده خواهد شد.
- توجه داشته باشید که کامپوننت‌های Class-based روش قدیمی ساخت کامپوننت در ری‌اکت هستند و شما باید از کامپوننت‌های Function-based به همراه Hook ها استفاده کنید.
- هدف این تمرین یادگیری و دور کردن ذهن شما از شرایط فعلی است. لطفا تمرین را خودتان انجام دهید.